

Documento Técnico: Sistema de Reservas FODUN

Autor: Cristian David Diez López
Rol: Analista Desarrollador .NET
Fecha: Mayo 2026
Versión: 1.0

Tabla de Contenidos

- 1. Introducción 1
- 2. Arquitectura de Software 1
 - 2.1 Estructura en Capas 1
 - 2.2 Patrón Repositorio 2
- 3. Modelo de Datos 2
- 4. Lógica de Negocio 3
- 5. Procedimientos Almacenados 4
- 6. Stack Tecnológico 5
- 7. Seguridad e Identidad 5
- 8. Despliegue 6
- 9. Datos de Prueba 7
- 10. Estructura de Directorios 7
- 11. Endpoints Principales 8
- 12. Conclusiones y Próximas Mejoras 8

1. Introducción

El Sistema de Reservas FODUN es una aplicación web desarrollada en .NET 8 que permite a los usuarios asociados del fondo consultar disponibilidad de sedes y apartamentos, calcular tarifas dinámicas según temporada y capacidad, y gestionar reservas en línea. La arquitectura implementa separación de capas, Entity Framework Core para acceso a datos y SQL Server para la lógica transaccional compleja.

2. Arquitectura de Software

2.1 Estructura en Capas

La solución está organizada en tres proyectos independientes siguiendo el patrón MVC y principios de arquitectura limpia:

Capa	Proyecto	Responsabilidad
-----	-----	-----
Dominio	SistemaReservas.Core	Entidades, interfaces, DTOs, lógica de negocio
Infraestructura	SistemaReservas.Data	DbContext, repositorios, Identity, migraciones
Presentación	SistemaReservas.Web	Controladores, vistas Razor, servicios de aplicación

****Principios:****

- Core no tiene dependencias externas
- Data implementa los contratos definidos en Core
- Web se comunica únicamente a través de interfaces

2.2 Patrón Repositorio

Cada entidad tiene un repositorio genérico (`IRepositoryBase<T>`) que expone operaciones CRUD. Los repositorios específicos heredan y agregan métodos de negocio:

- `IRepositorySede`: Consultas de sedes activas
- `IRepositoryTemporada`: Búsqueda de temporada por fecha
- `IRepositoryReserva`: Consultas por usuario y rango de fechas

3. Modelo de Datos

3.1 Relaciones entre Entidades (1:N)

1. **Sedes:** Posee múltiples Alojamientos, Reservas y Tarifas.
2. **Temporadas:** Define múltiples Tarifas según el periodo.
3. **Alojamientos:** Asociado a múltiples Tarifas y Detalles de Reserva.

4. **Reservas:** Contiene múltiples Detalles de Reserva y Servicios Adicionales.
5. **Servicios:** Relacionado con múltiples registros de Servicios por Reserva.

3.2 Esquema de Entidades Clave

Tabla	Propósito	Campos Clave
Sedes	Centros recreativos y unidades habitacionales	Id, Nombre, TipoSede, CapacidadTotal
Alojamientos	Unidades específicas de hospedaje	Id, Sedeld, CapacidadMaxima
Temporadas	Gestión de periodos de alta/baja demanda	Id, FechaInicio, FechaFin, TipoTemporada
Tarifas	Precios parametrizados por temporada	Id, AlojamientoId, TemporadaId, PrecioBase
Reservas	Cabecera transaccional de la reserva	Id, UsuarioId, Sedeld, FechaInicio, FechaFin

4. Lógica de Negocio

4.1 Flujo de Disponibilidad

1. Selección de sede, fechas y concurrencia por parte del usuario.
2. Ejecución del procedimiento
`sp_HabitacionesDisponibles_PorFechaYPersonas.`
3. Validación técnica de solapamientos, estados de reserva y capacidad instalada.
4. Presentación de opciones disponibles según criterios de búsqueda.

4.2 Cálculo de Tarifas

1. Identificación de la temporada correspondiente para cada pernoctación.
2. Extracción del precio base definido en la matriz de tarifas.

3. Aplicación de descuentos por días especiales (martes a jueves) si aplica.
4. Cálculo de recargos por excedente de capacidad e integración de servicios complementarios.

4.3 Estados de Reserva

Estado	Código	Descripción
Pendiente	0	Reserva creada a la espera de confirmación de pago.
Confirmada	1	Transacción validada satisfactoriamente.
CheckIn	2	Registro de ingreso del usuario en la sede.
CheckOut	3	Cierre de estadía y liberación de unidad.
Cancelada	4	Anulación de reserva (libera disponibilidad).

5. Procedimientos Almacenados

Se implementaron 4 SPs en T-SQL para optimizar operaciones transaccionales en la BD:

Procedimiento: sp_HabitacionesDisponibles_PorFecha

Parámetros de Entrada:

- @Sedeld (int)
- @FechaInicio / @FechaFin (datetime)

Parámetros de Salida:

- Listado de alojamientos disponibles sin traslape.

Procedimiento: sp_HabitacionesDisponibles_PorFechaYPersonas

Parámetros de Entrada:

- @Sedeld, @FechaInicio, @FechaFin
- @NumeroPersonas (int)

Parámetros de Salida:

- Alojamientos filtrados por capacidad y disponibilidad técnica.

Procedimiento: sp_ConsultarTarifas

Parámetros de Entrada:

- @AlojamientoId, @FechaInicio, @FechaFin, @NumeroPersonas

Parámetros de Salida:

- Desglose detallado de precios por noche y temporada.

Procedimiento: sp_CalcularTotalReserva

Parámetros de Entrada:

- @AlojamientoId, @NumeroPersonas, @FechaInicio, @FechaFin
- @IncluyeLavanderia (bit)

Parámetros de Salida:

- Totales calculados: Alojamiento, Servicios y Gran Total.

6. Stack Tecnológico

Componente	Tecnología	Versión
Framework	.NET Core	8.0
Acceso a Datos	Entity Framework Core	8.0
Base de Datos	Microsoft SQL Server	2022
Autenticación	ASP.NET Core Identity	8.0

7. Seguridad e Identidad

- **Autenticación:** ASP.NET Core Identity con contraseñas hasheadas (PBKDF2)
- **Autorización:** Controladores con atributo `[Authorize]` para rutas protegidas
- **Recuperación de Contraseña:** Token GUID enviado por email (MailKit + SMTP)
- **Datos Sensibles:** Almacenados en `appsettings.json` (no en código)

8. Despliegue

8.1 Requisitos

- Docker 20.10+
- Docker Compose 2.0+
- 2GB RAM mínimo
- Puerto 1433 (SQL Server)
- Puerto 5000 (Aplicación web)

8.2 Instrucciones

****Paso 1: Clonar/Extraer el proyecto****

```
```bash
cd Sistema_Reservas_net8
```
```

****Paso 2: Iniciar contenedores****

```
```bash
docker-compose up --build -d
```
```

****Paso 3: Aplicar datos semilla y SPs****

```
```bash
docker cp database/02_SeedData.sql sistema-reservas-db:/tmp/02_SeedData.sql
docker cp database/03_StoredProcedures.sql
sistema-reservas-db:/tmp/03_StoredProcedures.sql

docker exec sistema-reservas-db /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P
'Reservas@2024#Xk9!' -C -d SistemaReservas -i /tmp/02_SeedData.sql

docker exec sistema-reservas-db /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P
'Reservas@2024#Xk9!' -C -d SistemaReservas -i /tmp/03_StoredProcedures.sql
```
```

****Paso 4: Acceder a la aplicación****

Abrir navegador en `http://localhost:5000`

8.3 Verificación Rápida

```
```bash
```

# Confirmar BD creada

```
docker exec sistema-reservas-db sqlcmd -S localhost -U sa -P 'Reservas@2024#Xk9!' -C -Q
"SELECT COUNT(*) FROM SistemaReservas.sys.tables"
```

# Confirmar SPs creados

```
docker exec sistema-reservas-db sqlcmd -S localhost -U sa -P 'Reservas@2024#Xk9!' -C -Q
"SELECT name FROM SistemaReservas.sys.procedures WHERE name LIKE 'sp_%"

```

## 9. Datos de Prueba

La solución incluye:

- \*\*8 sedes:\*\* 6 recreativas + 2 apartamentos
- \*\*53 alojamientos:\*\* Variedad de capacidades (1-8 personas)
- \*\*8 temporadas:\*\* 4 en 2026, 4 en 2027
- \*\*424 tarifas:\*\* Combinaciones alojamiento × temporada
- \*\*1 servicio:\*\* Lavandería (\$18.000)

Crear usuario de prueba en el módulo de Registro para acceder a funcionalidades de reserva.

---

## 10. Estructura de Directorios

...

```
src/
├── SistemaReservas.Core/
│ ├── Entities/ (Modelos de dominio)
│ ├── Enums/ (Enumeradores)
│ ├── Interfaces/ (Contratos)
│ └── DTOs/ (Objetos de transferencia)
├── SistemaReservas.Data/
│ ├── Context/ (ApplicationDbContext)
│ ├── Repositories/ (Patrón repositorio)
│ └── Identity/ (ApplicationUser)
└── SistemaReservas.Web/
 ├── Controllers/ (Lógica de peticiones HTTP)
 └── Views/ (Vistas Razor)
```

└─ Services/ (Servicios de aplicación)  
└─ Program.cs (Configuración principal)

database/  
└─ 02\_SeedData.sql (Datos iniciales)  
└─ 03\_StoredProcedures.sql (Procedimientos)  
...  
  
---

## 11. API Endpoints Principales

Método	Ruta	Descripción
GET	/	Home / Landing Page
POST	/Cuenta/Registrar	Creación de perfil de usuario
POST	/Sede/BuscarDisponibilidad	Consulta de unidades libres
POST	/Reserva/Crear	Persistencia de nueva reserva

---

## 12. Conclusiones y Próximas Mejoras

**\*\*Estado actual:\*\*** Sistema funcional con lógica transaccional completa, autenticación segura y cálculo dinámico de tarifas.

**\*\*Mejoras futuras recomendadas:\*\***

- Integración con pasarela de pago (Stripe, PayU)
- Panel administrativo para gestión de temporadas y tarifas
- Reportes de ocupación y ingresos
- Notificaciones en tiempo real por WebSocket
- API REST para aplicaciones móviles
- Caché distribuido (Redis) para consultas frecuentes